



TRƯỜNG ĐẠI HỌC THƯƠNG MẠI
Khoa HTTT Kinh tế và THMĐT
Bộ môn Công nghệ thông tin

Chương 1

TỔNG QUAN VỀ KIỂM THỬ PHẦN MỀM





NỘI DUNG

- Khái niệm Phần mềm
- Vòng đời phát triển phần mềm
- Khái niệm Kiểm thử phần mềm
- Vai trò của Kiểm thử phần mềm



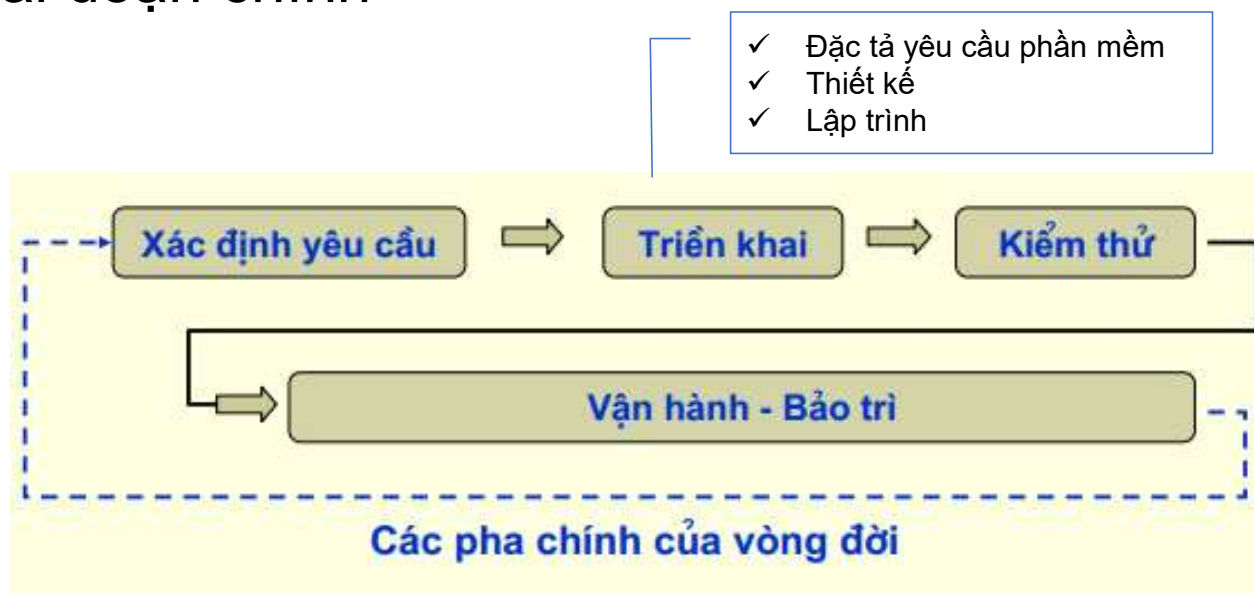
KHÁI NIỆM PHẦN MỀM

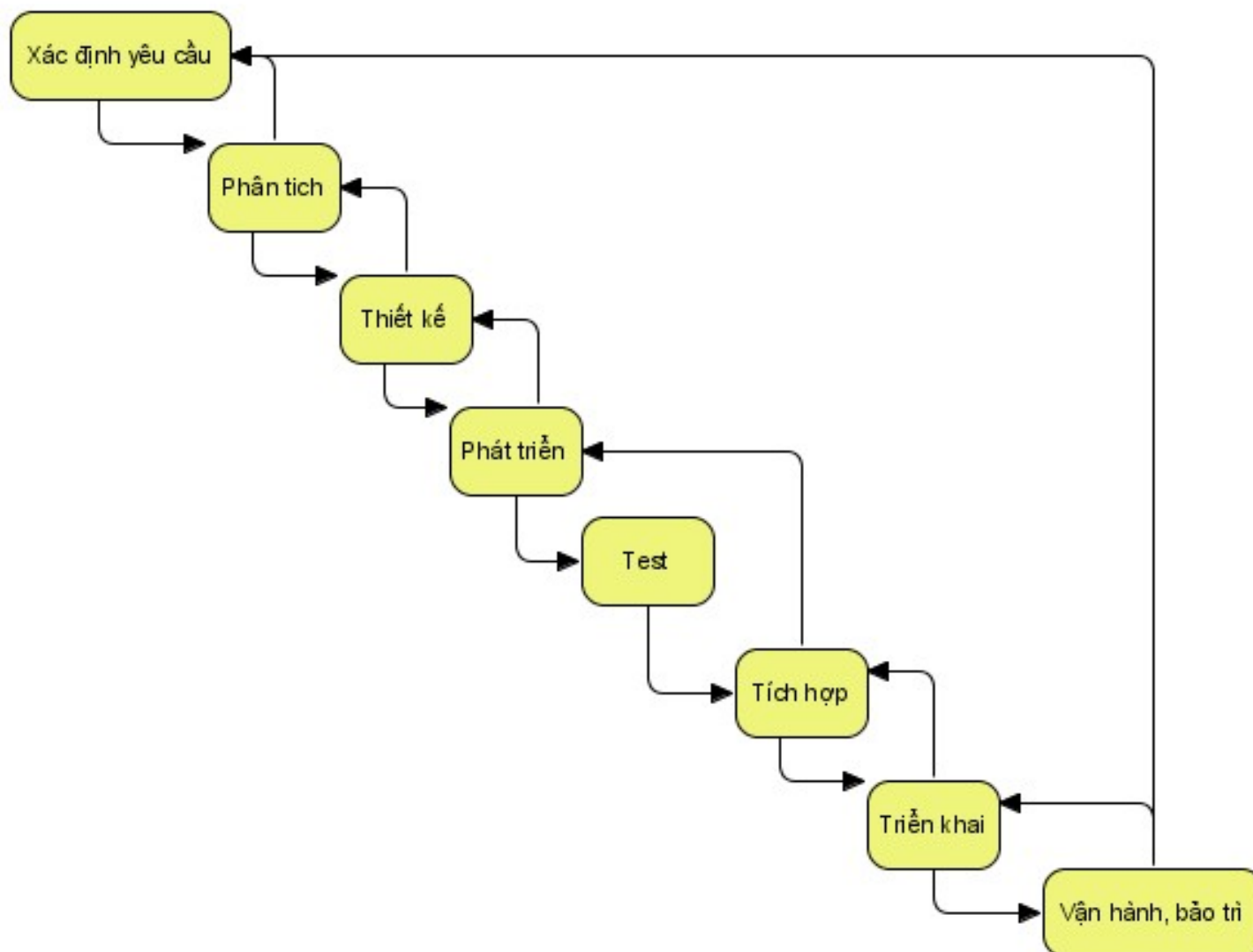
- Phần mềm (software) bao gồm:
 - Chương trình máy tính (mã nguồn, mã máy)
 - Cấu trúc dữ liệu
 - Cấu trúc làm việc: bộ nhớ trong
 - Cấu trúc lưu trữ: bộ nhớ ngoài
 - Tài liệu liên quan
 - Tài liệu hướng dẫn sử dụng (người dùng)
 - Tài liệu kỹ thuật (đội phát triển)



VÒNG ĐỜI PHÁT TRIỂN PHẦN MỀM

- Là các hoạt động từ khi được đặt hàng, phát triển, sử dụng đến khi loại bỏ nó
- Các giai đoạn chính







CÁC BƯỚC CHUNG NHẤT ĐỂ PHÁT TRIỂN PHẦN MỀM

- Xác định yêu cầu
 - Phân tích
 - Lập kế hoạch
 - Đặc tả yêu cầu
- Phát triển
 - Thiết kế
 - Mã hóa/lập trình
 - **Kiểm thử**
- Tiến hóa
 - Sửa lỗi, làm thích nghi
 - Bổ sung chức năng



YÊU CẦU KHÁCH HÀNG & ĐẶC TẢ PHẦN MỀM

- Phần mềm được viết để thực hiện yêu cầu của khách hàng
- Yêu cầu khách hàng là cơ sở để quyết định các đặc trưng mà phần mềm cần có
 - Khách hàng: *“tôi muốn việc tính tổng tiền đơn hàng được tự động hóa”*
- Dựa trên yêu cầu khách hàng để viết đặc tả yêu cầu phần mềm
 - Yêu cầu chức năng: *lập đơn hàng, thực hiện thanh toán, in hóa đơn,...*
 - Yêu cầu phi chức năng: *trên hóa đơn cần có mã hàng, tên hàng, đơn giá, số lượng, tổng tiền, tên thu ngân, có thể nhập mã hàng bằng máy đọc mã vạch hoặc nhập bằng bàn phím, có biểu tượng máy in để thực hiện chức năng in hóa đơn, ...*



Verification vs validation

- **Verification:**
 - "Are we building the **product right**?"
 - The software should conform to its specification
 - (Phần mềm nên nhất quán với đặc tả yêu cầu)
- **Validation:**
 - "Are we building the **right product**?"
 - The software should do what the user really requires
 - (Phần mềm nên thỏa mãn yêu cầu của người dùng)

Verification: cần có tri thức về phát triển phần mềm
Validation: cần có tri thức của chuyên gia miền ứng dụng



Verification & Validation (V&V)

- V&V phải được áp dụng ở mọi giai đoạn của tiến trình làm phần mềm
- Cần thực hiện kiểm chứng (verification) trước thẩm định (validation)
 - Nếu thực hiện thẩm định trước mà phát hiện lỗi thì không biết lỗi do đặc tả hay thiết kế, lập trình
- Mục tiêu
 - Phát hiện lỗi;
 - Đánh giá xem hệ thống có hữu ích (thỏa mãn yêu cầu người dùng) và có khả năng sử dụng trong môi trường vận hành không.

💡 Tóm tắt:

- Verification = Xây dựng đúng sản phẩm theo tài liệu.
- Validation = Xây dựng sản phẩm đúng với nhu cầu.



Các mức độ của lỗi

1	Nhẹ	Lỗi chính tả
2	Vừa	Hiểu lầm hoặc thừa thông tin
3	Khó chịu	Tên bị thiếu, cắt chữ hoặc hóa đơn có giá trị 0.0 đồng
4	Bực mình	Vài giao dịch không được xử lý
5	Nghiêm trọng	Mất giao dịch
6	Rất nghiêm trọng	Xử lý giao dịch sai
7	Cực kỳ nghiêm trọng	Lỗi rất nghiêm trọng xảy ra thường xuyên
8	Quá quắt	Hủy hoại cơ sở dữ liệu
9	Thảm họa	Hệ thống bị tắt
10	Dịch họa	Thảm họa lây lan

Theo Bor84



Sai lầm về đầu vào, đầu ra

Loại	Các trường hợp
dữ liệu đầu vào	dữ liệu đầu vào đúng nhưng không được chấp nhận
	dữ liệu đầu vào sai nhưng được chấp nhận
	mô tả sai hoặc thiếu
	tham số sai hoặc thiếu
dữ liệu đầu ra	khuôn dạng sai
	kết quả sai
	kết quả tính toán đúng nhưng không đúng thời điểm (quá sớm hoặc quá muộn)
	kết quả không đầy đủ hoặc thiếu
	kết quả giả tạo
	văn phạm/chính tả
	các trường hợp khác



Sai lầm về logic

thiếu trường hợp
lập thừa trường hợp
điều kiện cực đoan bị bỏ qua
thể hiện sai
thiếu điều kiện
điều kiện ngoại lai
kiểm thử sai biến
việc lập của chu trình không đúng
phép toán sai (chẳng hạn dùng $<$ cho $\leq$)



Sai lầm về tính toán

thuật toán sai
thiếu tính toán
toán hạng sai
sai về dấu ngoặc
chưa đủ độ chính xác (làm tròn hoặc cắt đuôi)
hàm đi kèm sai



Sai lầm về giao diện phần mềm

xử lý gián đoạn sai (trong các hệ thống nhúng)
--

thời gian vào ra (time out)

gọi sai thủ tục

gọi đến một thủ tục không tồn tại

tham số không khớp (mismatch) (chẳng hạn về kiểu và số)

kiểu không tương thích

bao hàm thừa



KHÁI NIỆM KIỂM THỬ PHẦN MỀM

- Kiểm thử phần mềm (software testing) là một hoạt động kiểm tra, đánh giá chất lượng của phần mềm
- Kiểm thử là quy trình kiểm chứng để đánh giá và tăng cường chất lượng của sản phẩm phần mềm
- Nhằm phát hiện lỗi hoặc bất cứ vấn đề gì về sản phẩm
 - Liên quan đến các khái niệm: **lỗi, sai sót, thất bại, sự cố**
- **Có thể chỉ ra lỗi, không thể khẳng định không còn lỗi**
 - Có thể khẳng định hết lỗi bằng kiểm thử vét cạn, nhưng cách này không khả thi trên thực tế

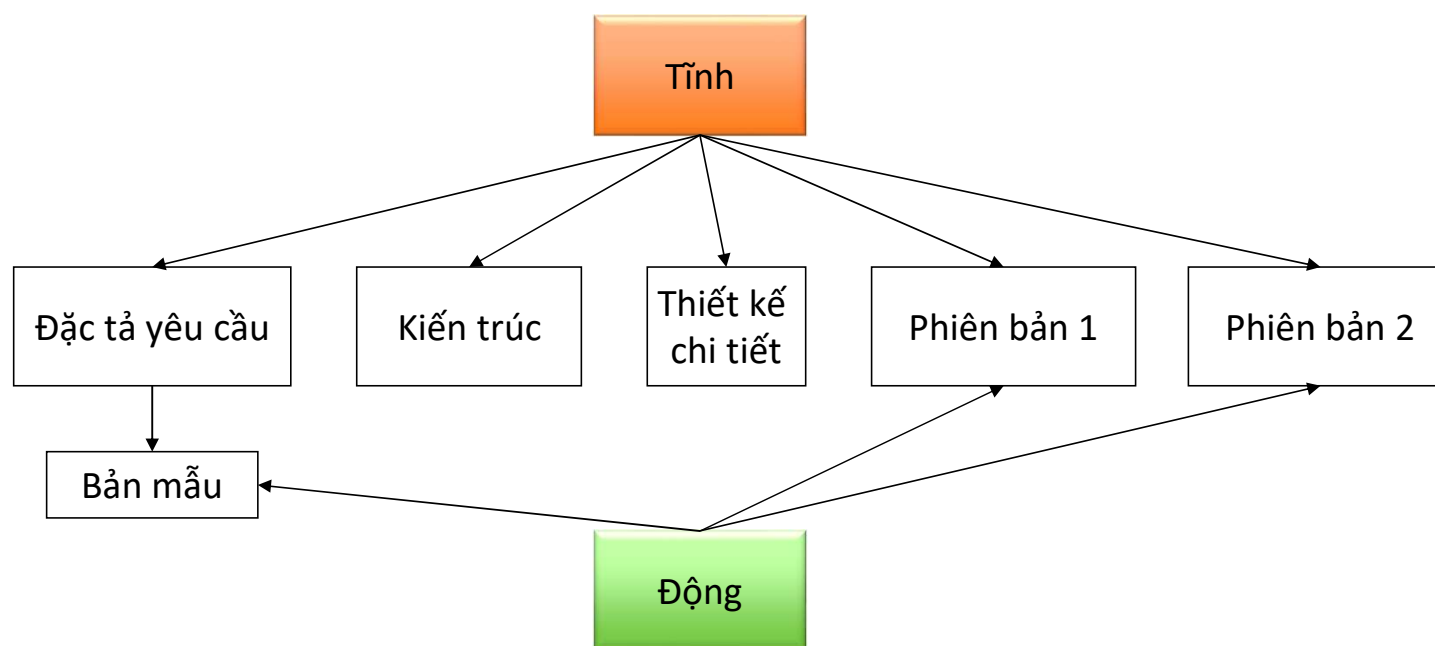


Static and dynamic testing

- **Static** testing
 - Kiểm tra (review) tài liệu đặc tả, tài liệu thiết kế, mã nguồn trước khi chạy
 - Có thể được hỗ trợ bởi công cụ phân tích mã nguồn
- **Dynamic** testing
 - Kiểm tra bằng cách chạy chương trình
 - Hệ thống được thực hiện với dữ liệu kiểm thử và hành vi của hệ thống được quan sát.



Kiểm thử tĩnh và động





Static and dynamic testing

- **Static và dynamic** testing là bổ sung cho nhau
- Cả hai nên được thực hiện lặp đi lặp lại trong quy trình
- Static testing có thể kiểm tra tính nhất quán (**conformance**) của phần mềm so với đặc tả
 - Không phải là nhất quán với yêu cầu của người dùng
- Static testing không thể kiểm tra một số yêu cầu phi chức năng
 - Vd, hiệu năng, khả năng sử dụng



Static and dynamic testing

- **Static và dynamic** testing là bổ sung cho nhau
- Cả hai nên được thực hiện lặp đi lặp lại trong quy trình
- Dynamic testing:
 - Hầu như không thể kiểm tra trên tất cả các dữ liệu vào có thể
 - Chỉ có thể chọn một tập con các dữ liệu vào để thực thi chương trình -> các ca kiểm thử (**test cases**)

Học phần này tập trung vào hoạt động **Dynamic Testing**



Mục tiêu của kiểm thử

- Validation testing
 - Chỉ ra rằng phần mềm thỏa mãn yêu cầu
 - Chỉ ra rằng hành vi của hệ thống là hành vi được mong đợi
- Defect testing
 - Phát hiện lỗi
 - Ví dụ, chương trình không nhất quán với đặc tả
 - Ca kiểm thử thành công là ca kiểm thử trong đó hệ thống hoạt động sai
 - Lộ ra vấn đề của hệ thống



VAI TRÒ CỦA KIỂM THỬ

- Đánh giá chất lượng phần mềm
- Góp phần cải tiến chất lượng phần mềm bằng cách thực hiện lặp đi lặp lại việc tìm lỗi để sửa lỗi

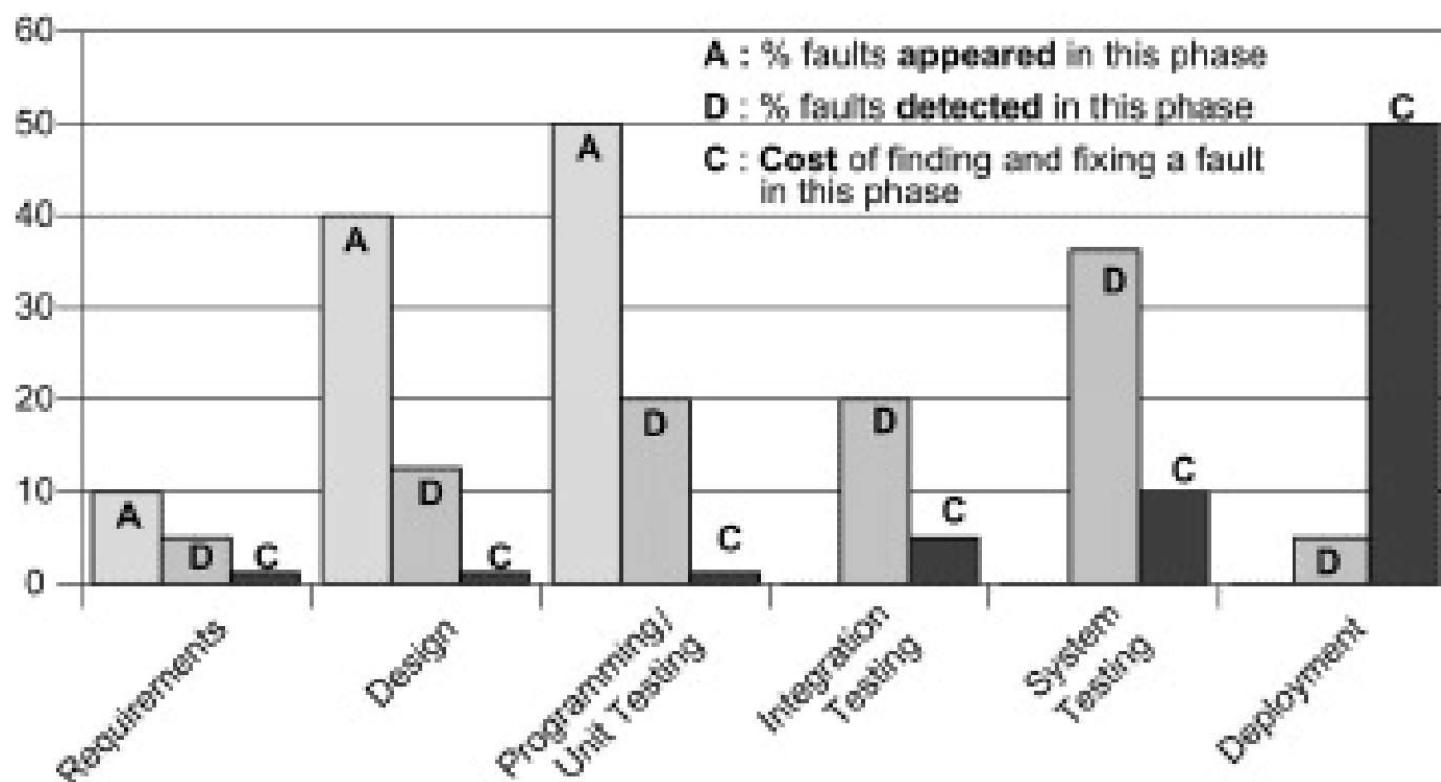
Chi phí dành cho kiểm thử tiêu hao trên 50% tổng giá phát triển phần mềm





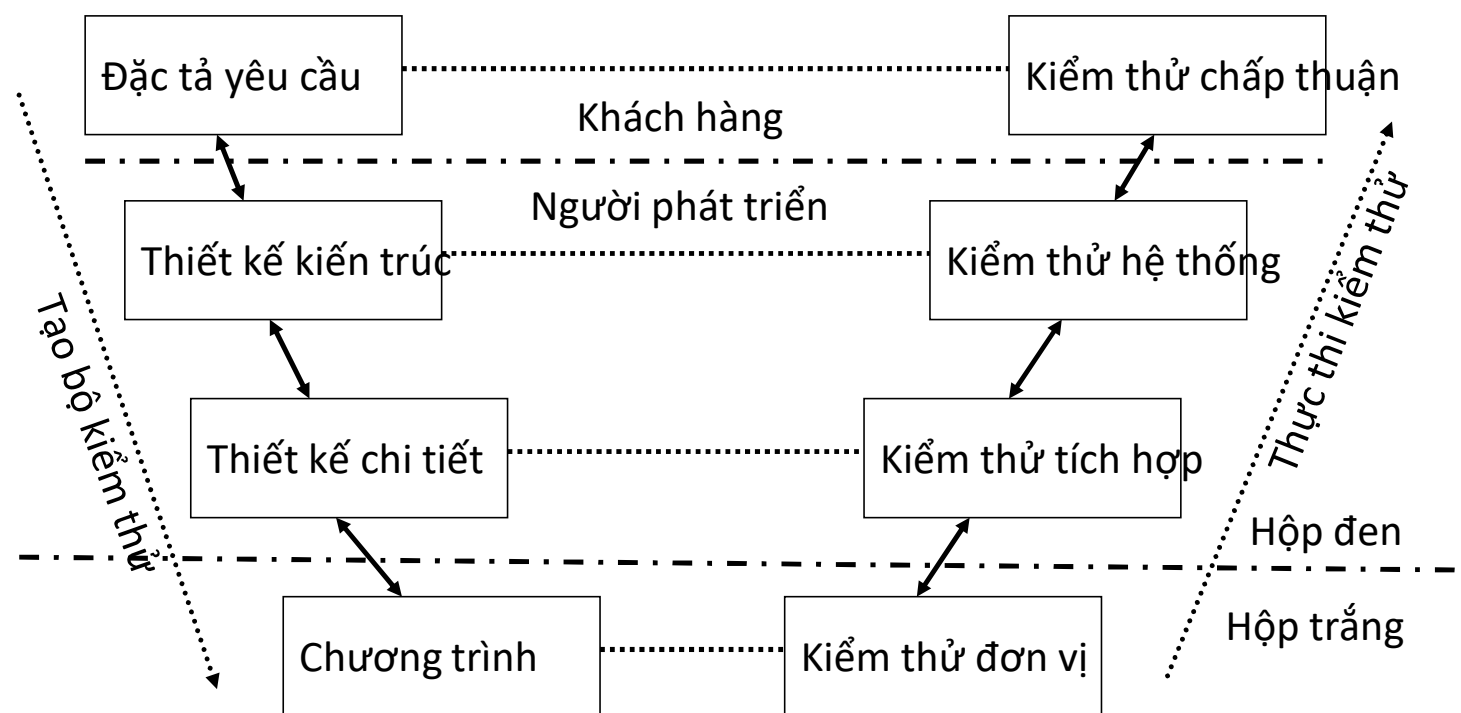
Vai trò của kiểm thử

- Kiểm thử càng muộn, chi phí càng lớn





Kiểm thử trong mô hình chữ V





CS1

- Tìm hiểu về các mức kiểm thử, lấy ví dụ cụ thể trên sản phẩm cụ thể (ví dụ: congdaotao.tmu.edu.vn)
- Tìm hiểu và trình bày về kiểm thử hộp đen, hộp trắng



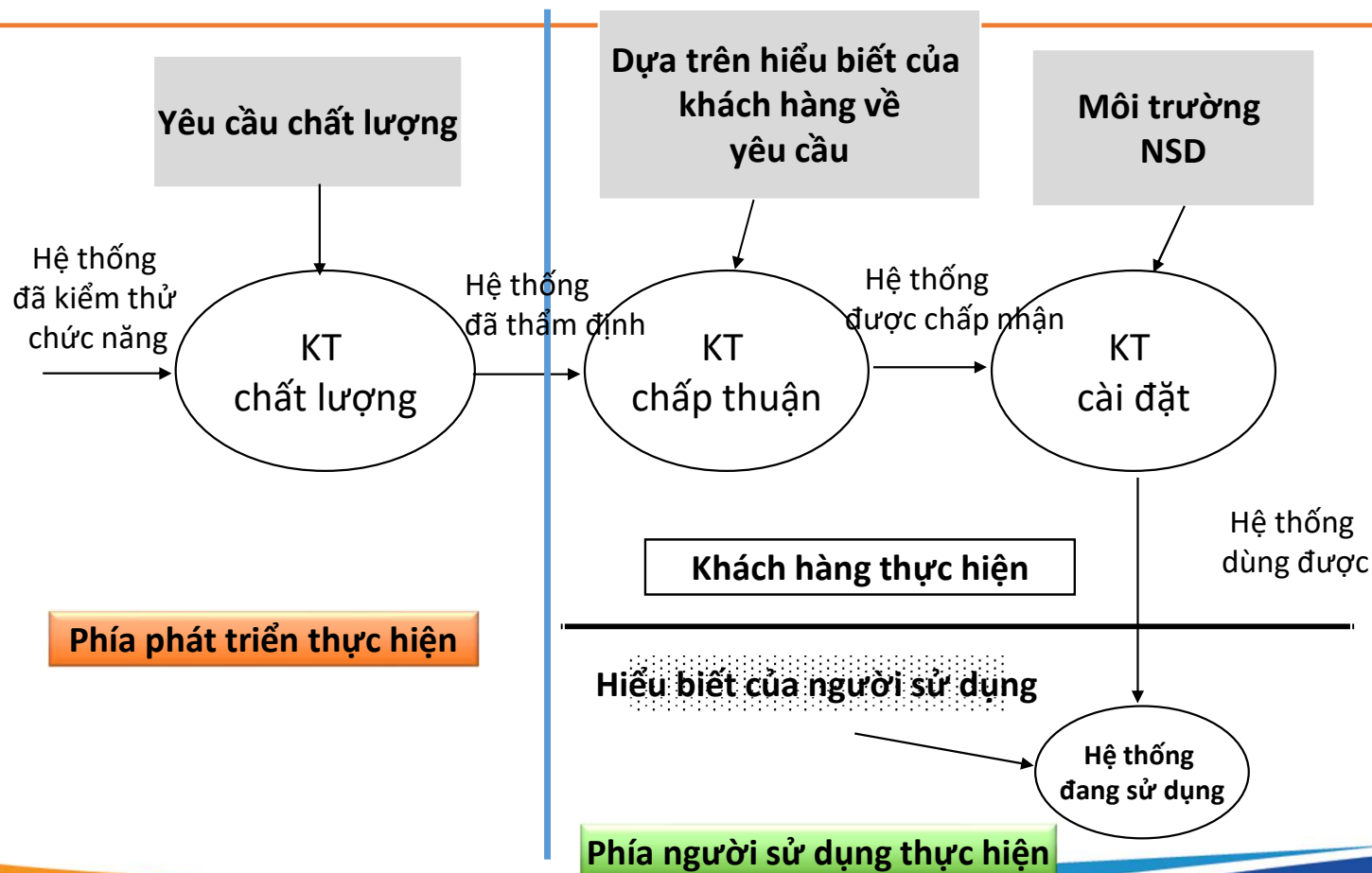
Các mức kiểm thử

- Đơn vị
 - Tìm lỗi trong từng đơn vị
- Tích hợp
 - Tìm lỗi khi ghép các đơn vị
- Hệ thống
 - Tìm lỗi khi hệ thống đã tích hợp xong, trước khi phát hành, chuyển giao
- Chấp thuận
 - Người sử dụng dùng thử xem hệ thống đáp ứng đúng mong muốn chưa.
 - Còn gọi là kiểm thử alpha.



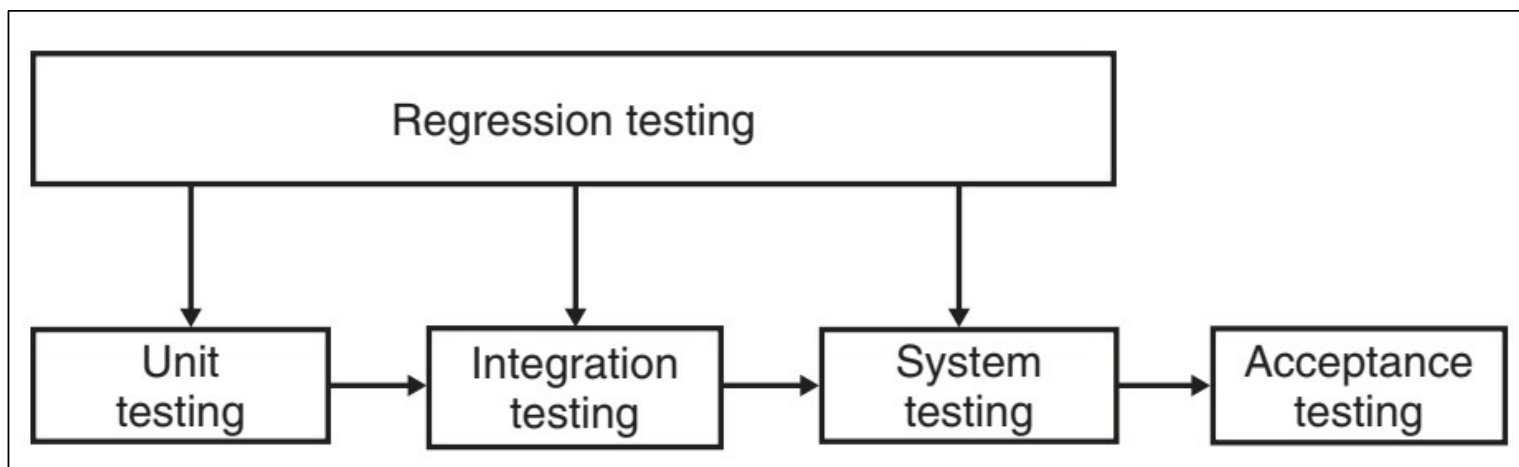


Các mức kiểm thử (tiếp)





Các mức kiểm thử





Kiểm thử và gỡ lỗi (debugging)

- Kiểm thử khiếm khuyết
 - Khẳng định có lỗi
- Gỡ lỗi (debugging)
 - Định vị và sửa lỗi
- Gỡ lỗi thông thường phải lập giả thuyết về hành vi của chương trình và kiểm tra các giả thuyết này để tìm lỗi

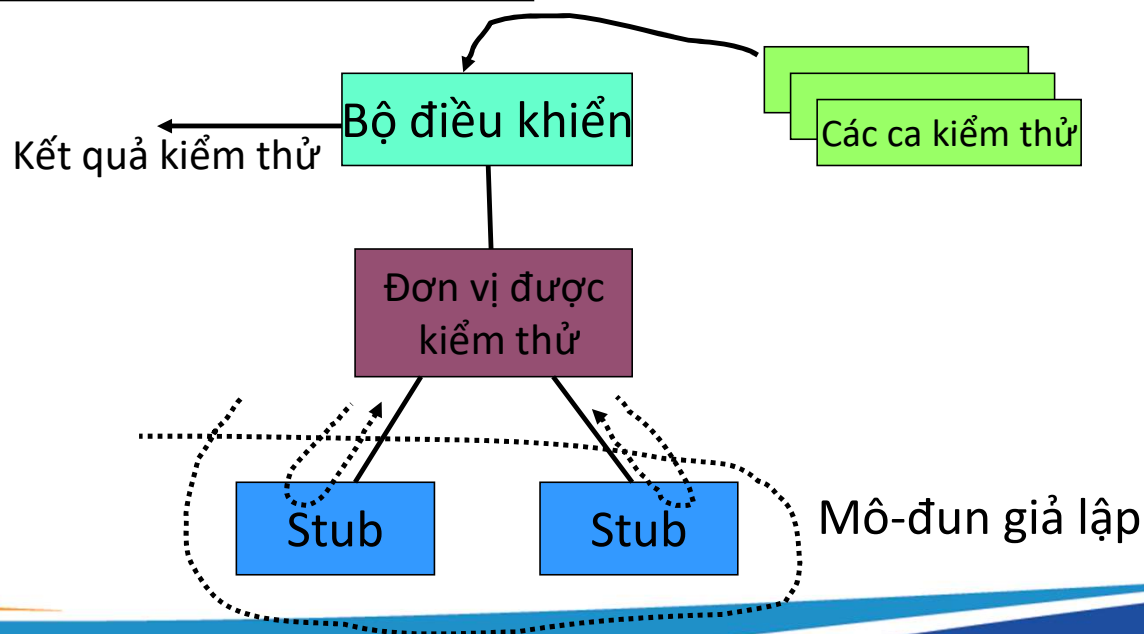


Kiểm thử đơn vị

Mục đích: Tìm sự khác biệt giữa đặc tả và cài đặt của đơn vị

Đơn vị: các lớp, hàm, đối tượng, gói, mô-đun

Môi trường kiểm thử đơn vị:





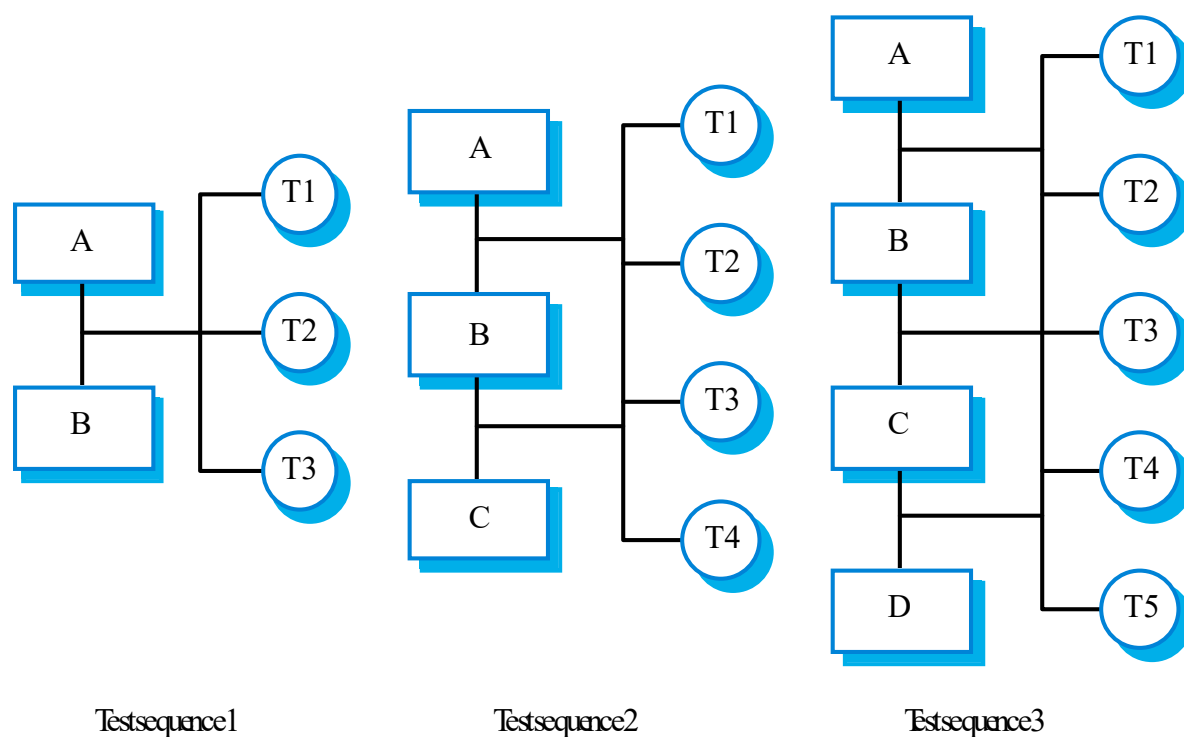
Kiểm thử tích hợp

- Mục tiêu:
 - Phát hiện vấn đề khi ghép các mô-đun/thành phần với nhau
- Các vấn đề
 - Bên trong: giữa các thành phần
 - Gọi: call/message passing/...
 - Tham số: kiểu, số lượng, thứ tự, giá trị
 - Kết quả trả về: ai, kiểu, trình tự
 - Bên ngoài:
 - Ngắt (wrong handler?)
 - Thời gian vào ra
 - Tương tác



Incremental integration testing

Kiểm thử tích hợp gia tăng





Kiểm thử hệ thống

- Liên quan đến các yếu tố bên ngoài hệ thống
- Không chỉ là kiểm tra chức năng
 - Khả dụng (usability)
 - Giao diện, thông báo, dễ học, dễ nhớ..
 - Hiệu năng
 - Khả năng đáp ứng/Tìm khả năng đáp ứng
 - Tài nguyên sử dụng



Khi nào nên dừng kiểm thử

- Hết thời gian, hết ngân sách
- Đạt mức độ bao phủ mong muốn
- Đạt tần suất hỏng hóc mong muốn



Kiểm thử chấp thuận

- Mục đích: kiểm tra sự hài lòng của người sử dụng
- Cơ sở: mong muốn của người dùng (không xét đến tài liệu đặc tả)
- Môi trường: thật
- Người thực hiện: bởi và cho người sử dụng
- Các ca kiểm thử:
 - Sử dụng lại từ kiểm thử hệ thống
 - Do người dùng thiết kế



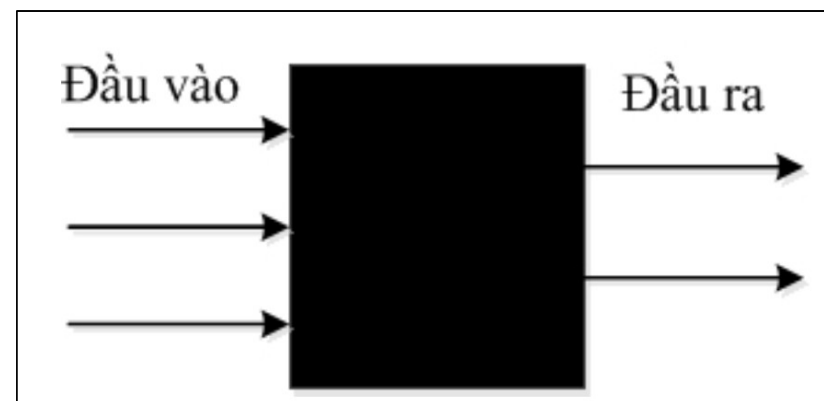
Kiểm thử hồi qui

- Khi một hệ thống được chỉnh sửa (sửa lỗi, thêm/bớt chức năng,..) toàn bộ bộ kiểm thử cần phải chạy lại
 - Đảm bảo các tính năng đang hoạt động tốt không bị ảnh hưởng bởi chỉnh sửa
- Kiểm thử lại tự động trước khi lưu thay đổi vào kho (repo.)
- Cần các chiến lược kiểm thử tăng dần với hệ thống lớn



Kiểm thử hộp đen

- Còn gọi là kiểm thử hàm, kiểm thử chức năng
- Tập trung vào hành vi vào/ra. Với đầu vào đã biết ra có thể đoán/tính đầu ra, rồi kiểm tra chương trình có tạo kết quả như ta đoán/tính.
 - Không thể kiểm thử hết các bộ dữ liệu đầu vào



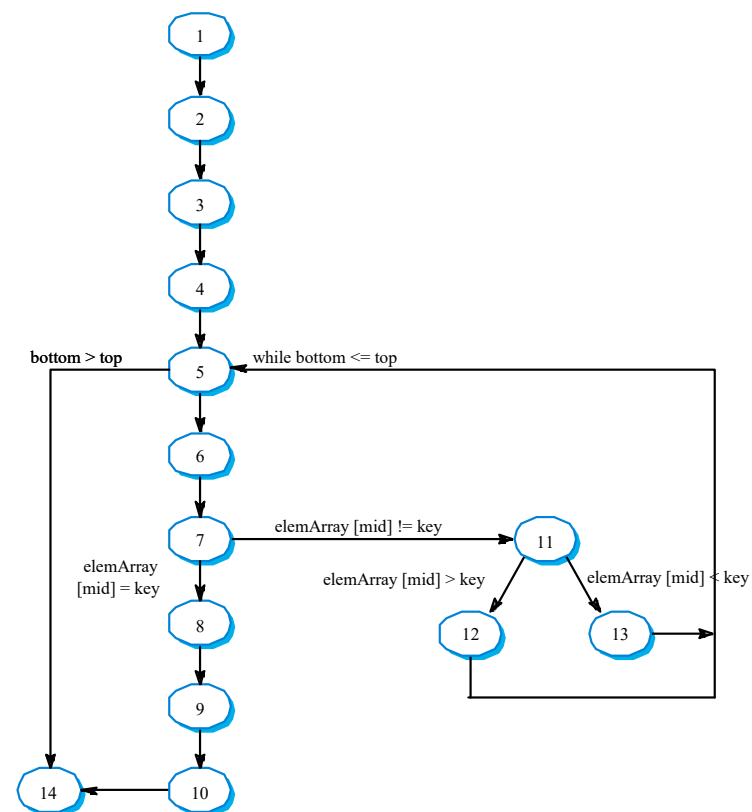
➔ Giảm số lượng ca kiểm thử bằng việc chia không gian đầu vào thành các miền tương đương
Chọn một ca kiểm thử từ mỗi miền tương đương này (chi tiết trong chương 3)



Kiểm thử hộp trắng

Còn gọi là kiểm thử cấu trúc, kiểm thử logic
Các tiêu chuẩn bao phủ:

- Lệnh
 - Mọi lệnh đều được thử
- Vòng lặp
 - 0, 1, >1 lần
- Đường đi
 - Tất cả các khả năng chạy của chương trình
- Nhánh (if, while, ..)
 - Biểu thức điều kiện được thử với cả True và False
 - Các nhánh đều được chạy ít nhất một lần





So sánh kiểm thử hộp trắng và hộp đen

Hộp trắng

- Số đường đi nhiều khi là vô hạn
- Kiểm tra những gì đã làm, không phải những gì cần được làm
- Không phát hiện được ca kiểm thử còn thiếu
- Cần kiến thức về chi tiết thiết kế và hiện thực bên trong
- Theo góc nhìn hiện thực
- Kiểm thử dựa vào phủ các lệnh, phủ các nhánh, phủ các điều kiện con, ...

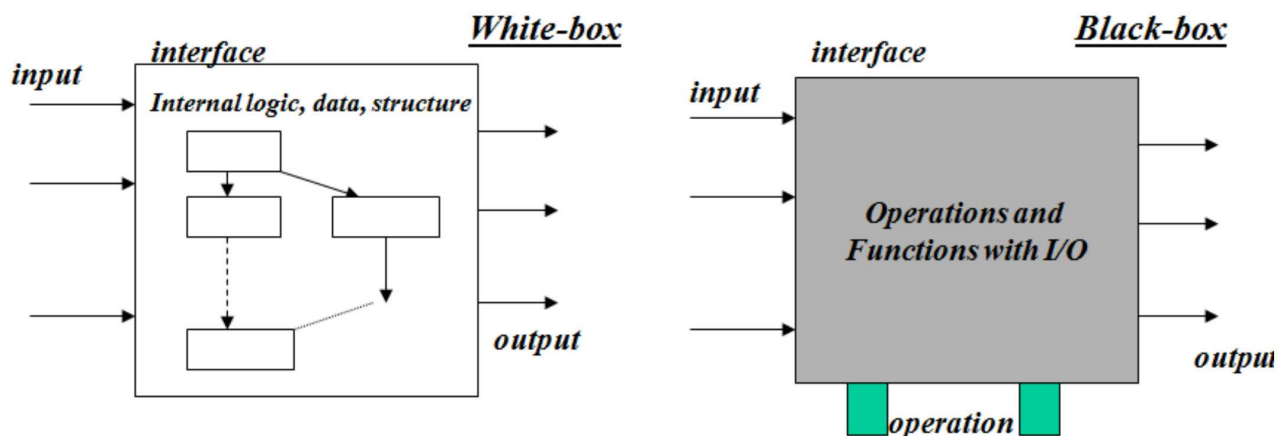
Hộp đen

- Dễ bùng nổ tổ hợp về số ca kiểm thử (dữ liệu đúng và dữ liệu sai)
- Thường không chắc ca kiểm thử này có phát hiện được lỗi cụ thể kia hay không
- Thích hợp cho kiểm thử đơn vị và tích hợp.
- Không cần kiến thức về chi tiết thiết kế và hiện thực bên trong
- Theo góc nhìn sử dụng

- Cần cả hai
- Kiểm thử hộp trắng và hộp đen là hai thái cực của kiểm thử
- Việc lựa chọn ca kiểm thử nằm giữa và phụ thuộc vào
 - ✓ Số đường đi logic có thể
 - ✓ Tính chất của dữ liệu đầu vào
 - ✓ Khối lượng tính toán
 - ✓ Độ phức tạp của cấu trúc dữ liệu và giải thuật
- Hai kỹ thuật là bổ sung cho nhau.



So sánh kiểm thử hộp trắng và hộp đen



Kiểu kiểm thử	Kỹ thuật kiểm thử được dùng
Unit Testing	White Box, Black Box
Integration Testing	Black Box, White Box
Functional Testing	Black Box
System Testing	Black Box
Acceptance Testing	Black Box



Thuật ngữ (tiếp)

- Test case: ca kiểm thử
- Test suite: bộ kiểm thử, tập hợp các test cases
- Test oracle: bất kỳ thực thể gì mà cho chúng ta biết về đầu ra mong đợi của test cases
- System Under Test (SUT): hệ thống được kiểm thử
- Test driver: chương trình dẫn dắt thực hiện bộ kiểm thử



Một số nguyên tắc cho kiểm thử

- Kiểm thử đòi hỏi tính độc lập: lập trình viên nên tránh việc kiểm thử các thành phần phần mềm mình viết
- Chương trình có thể chứa các lỗi do lập trình viên hiểu sai về yêu cầu phần mềm
- Phải thiết kế đủ các ca kiểm thử cho cả hai trường hợp: dữ liệu đầu vào hợp lệ và dữ liệu đầu vào không hợp lệ
- Nên bắt đầu từ những thành phần nhỏ, đơn giản rồi đến thành phần lớn hơn



Đánh giá một số nhận định

- Ta có thể vét cạn mọi hoạt động kiểm thử
- Ta có thể tìm tất cả các lỗi nếu thực hiện tốt hoạt động kiểm thử
- Tập các ca kiểm thử tốt phải chứa rất nhiều ca kiểm thử
- Ca kiểm thử tốt là ca kiểm thử phức tạp



Nhiều công cụ hỗ trợ các loại kiểm thử

- Kiểm thử đơn vị: **Achoo, JUnit, Pex/Moles, PyUnit**
- Tự động kiểm thử: **TestComplete**
- Kiểm thử hiệu năng và tải: **JMeter**
- Kiểm thử giao diện đồ họa (GUI): **Abbot, Guitar**
- Kiểm thử tổ hợp: **AETG, FireEye**
- Kiểm thử dựa trên mô hình: **Spec Explorer**
- Phân tích bao phủ: **Corbertura**
- Quản lý lỗi (defects): **Bugzilla**
- **Test API: Postman**



Tổng kết

- Kiểm thử là hoạt động đánh giá chất lượng phần mềm và tìm lỗi phần mềm
 - Kiểm thử góp phần tăng chất lượng phần mềm
 - Chi phí cho kiểm thử chiếm trên 50% chi phí phát triển
 - Cần tự động hóa nhiều nhất có thể hoạt động kiểm thử để giảm thiểu chi phí, đặc biệt cho kiểm thử hồi quy
 - Các mức kiểm thử: đơn vị, tích hợp, hệ thống, chấp nhận
 - Các loại kiểm thử: hộp đen, hộp trắng
- 



Bài tập

- Hai đoạn code có defect/error?
- Nếu có giải thích lý do và cách sửa
- Lấy ví dụ các TCs có thể/không thể phát hiện lỗi

```
/**
 * Find last index of element
 *
 * @param x array to search
 * @param y value to look for
 * @return last index of y in x; -1 if absent
 * @throws NullPointerException if x is null
 */
public int findLast (int[] x, int y)
{
    for (int i=x.length-1; i > 0; i--)
    {
        if (x[i] == y)
        {
            return i;
        }
    }
    return -1;
}
// test: x = [2, 3, 5]; y = 2; Expected = 0
// Book website: FindLast.java
// Book website: FindLastTest.java
```

```
/**
 * Find last index of zero
 *
 * @param x array to search
 *
 * @return last index of 0 in x; -1 if absent
 * @throws NullPointerException if x is null
 */
public static int lastZero (int[] x)
{
    for (int i = 0; i < x.length; i++)
    {
        if (x[i] == 0)
        {
            return i;
        }
    }
    return -1;
}
// test: x = [0, 1, 0]; Expected = 2
// Book website: LastZero.java
// Book website: LastZeroTest.java
```



Câu hỏi ôn tập

1. Khái niệm phần mềm, vòng đời phần mềm, kiểm thử phần mềm?
2. Các giai đoạn chính trong vòng đời phát triển phần mềm là gì?
3. Các hoạt động chung nhất của mọi tiến trình phát triển phần mềm là gì?
4. Khái niệm kiểm thử, vai trò của kiểm thử, mục tiêu của kiểm thử
5. Phân biệt verification và validation
6. Phân biệt kiểm thử tĩnh và kiểm thử động
7. Phân biệt các mức kiểm thử: kiểm thử đơn vị, kiểm thử tích hợp, kiểm thử hệ thống, kiểm thử chấp nhận, kiểm thử hồi quy
8. Phân biệt kiểm thử chức năng (hộp đen) và kiểm thử cấu trúc (hộp trắng)
9. Các khái niệm liên quan: ca kiểm thử, bộ kiểm thử, ...



Bài tập ở lớp và về nhà

Mỗi sinh viên chương trình đơn giản bằng ngôn ngữ C, Java hay tùy ý (các chương trình này sẽ được sử dụng nhiều lần trong suốt học kỳ) trong số các chương trình dưới đây:

1. Nhập vào hai số nguyên, thực hiện phép chia và in ra kết quả phép chia
2. Chương trình tìm số lượng phần tử bằng 0 trong một dãy số nguyên
3. Chương trình tam giác
4. Chương trình mô phỏng hoạt động bán hàng ở cửa hàng bán lẻ văn phòng phẩm
5. Chương trình mô phỏng máy ATM ngân hàng với các chức năng Rút tiền, Chuyển tiền, Nạp tiền, Ván tin số dư
6. Chương trình tìm một phần tử trong dãy số nguyên